DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 3

Student Name: Probal Dhali **UID:** 24BAI71013
Branch: AIT-CSE (AI ML) **Section/Group:** B
Semester: 5TH **Date of Performance:**
Subject Name: Competitive Coding – II **Subject Code:** 24CSP-305

Experiment No: 3.1

Aim:
To implement a Queue data structure using two Stacks.

Objective:

1. To understand the FIFO principle of a queue.
2. To implement Queue operations using Stacks.
3. To perform push, pop, peek, and empty operations.
4. To understand how two Stacks can simulate a Queue.

Problem Statement:
Implement a first-in, first-out (FIFO) Queue using only two Stacks. The implemented Queue should support the operations push(), pop(), peek(), and empty().

LeetCode Link: https://leetcode.com/problems/implement-queue-using-stacks/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Create two Stacks: stack1 and stack2.
2. Use stack1 to insert elements.
3. Move elements from stack1 to stack2 when needed.
4. Use stack2 to remove or view the front element.
5. Check whether both stacks are empty.

Code:

```
class MyQueue {

private:

    stack<int> input;

    stack<int> output;

public:

    MyQueue() {

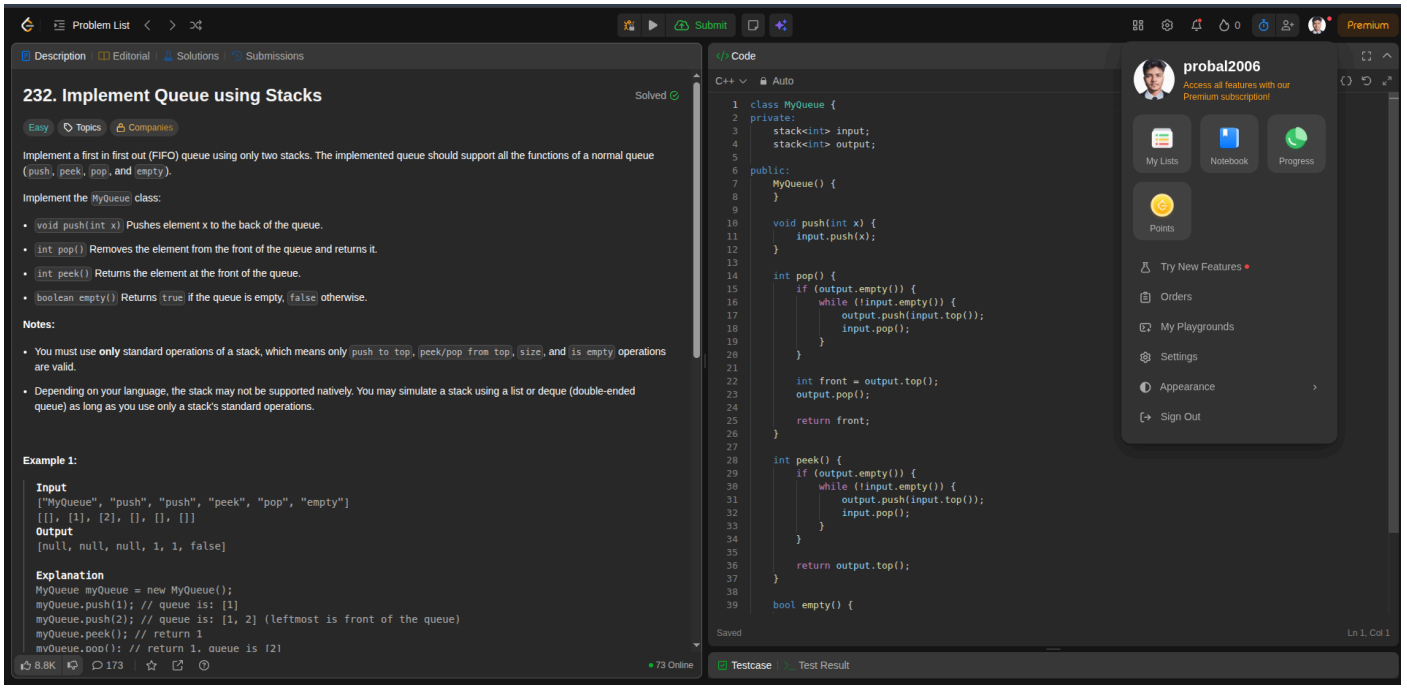
    }

    void push(int x) {

        input.push(x);
```

```
    }
    int pop() {
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }
        int front = output.top();
        output.pop();
        return front;
    }
    int peek() {
        if (output.empty()) {
            while (!input.empty()) {
                output.push(input.top());
                input.pop();
            }
        }
        return output.top();
    }
    bool empty() {
        return input.empty() && output.empty();
    }
};
```

Output:



Observations/Outcome: The Queue was successfully implemented using two Stacks while maintaining the FIFO order.

Time Complexity: O(1) amortized for push, pop, and peek.

Space Complexity: O(n)

Learning outcomes:

1. Learned the FIFO concept of Queue.
2. Learned the LIFO concept of Stack.
3. Learned how two Stacks can implement a Queue.
4. Understood push, pop, and peek operations.
5. Learned amortized time complexity.

Experiment No: 3.2

Aim:
To find the largest rectangular area in a histogram using a Stack.

Objective:

1. To understand the histogram rectangle problem.
2. To use a Stack for efficient processing.
3. To find the maximum rectangular area.
4. To optimize the solution compared to the brute-force approach.

Problem Statement:
Given an array of integers heights representing the heights of histogram bars, find the area of the largest rectangle that can be formed within the histogram.

LeetCode Link: https://leetcode.com/problems/largest-rectangle-in-histogram/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Create an empty Stack to store indices.
2. Traverse each histogram bar.
3. If the current bar is smaller, calculate the area using previous heights.
4. Update the maximum area.
5. Push the current index into the Stack.
6. Return the maximum area.

Code:

```
class Solution {

public:

    int largestRectangleArea(vector<int>& heights) {

        stack<int> st;

        int maxArea = 0;

        for (int i = 0; i <= heights.size(); i++) {

            int currentHeight;

            if (i == heights.size()) {

                currentHeight = 0;

            } else {

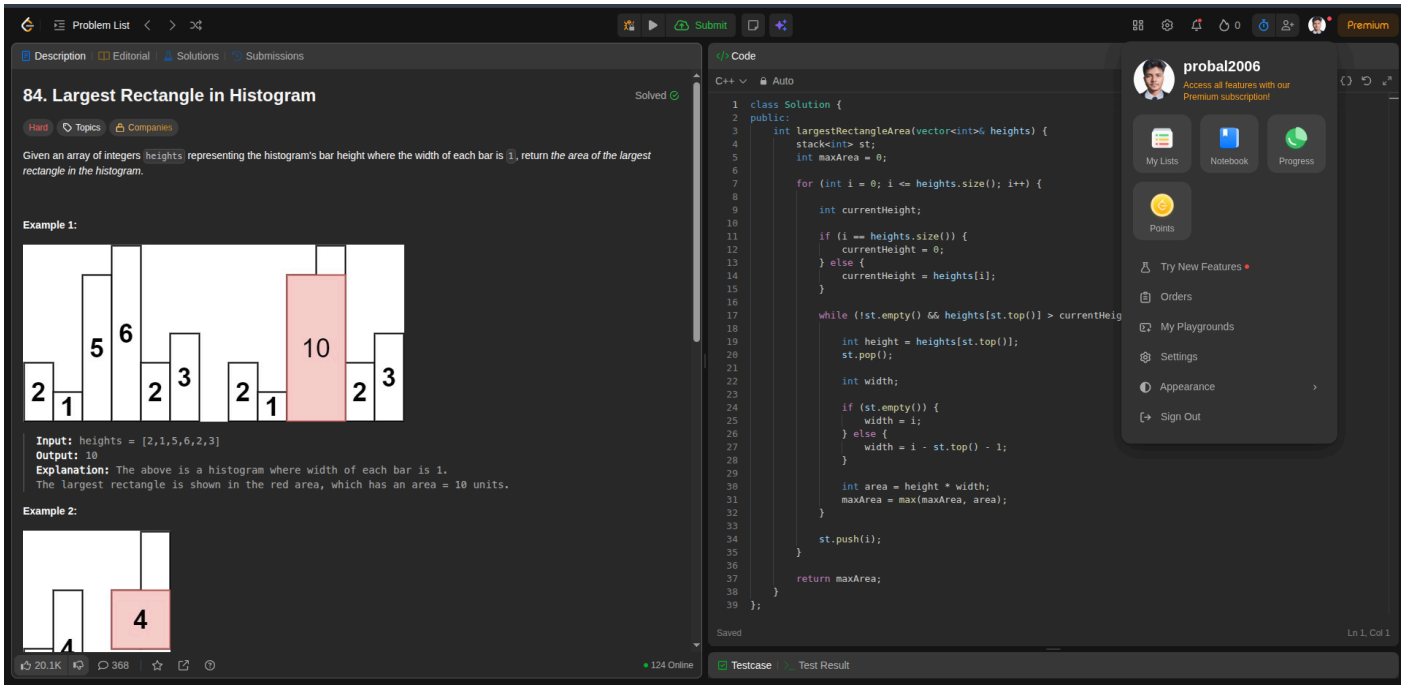
                currentHeight = heights[i];

            }

            while (!st.empty() && heights[st.top()] > currentHeight) {
```

```
            int height = heights[st.top()];
            st.pop();
            int width;
            if (st.empty()) {
                width = i;
            } else {
                width = i - st.top() - 1;
            }
            int area = height * width;
            maxArea = max(maxArea, area);
        }
        st.push(i);
    }
    return maxArea;
  }
};
```

Output:



Observations/Outcome: The program successfully finds the largest possible rectangular area in the histogram using a Stack.

Time Complexity: O(n)

Space Complexity: O(n)

Learning outcomes:

1. Learned how to use a Monotonic Stack.
2. Learned how to calculate the area of histogram rectangles.
3. Understood how a stack helps optimize array problems.
4. Learned to find the previous smaller element.
5. Improved understanding of `O(n)` algorithms.